

Investigación de APIs para Pasarelas de Pago en el Contexto del Proyecto WiseTrip

Gabriela Melo Gualteros

Fundamentos de Ingeniería de Software

Proyecto WiseTrip

Agosto de 2026

Investigación de APIs para Pasarelas de Pago en el Contexto del Proyecto WiseTrip

Introducción

WiseTrip es una plataforma web orientada a la planificación personalizada de viajes según el presupuesto y las preferencias del usuario, centralizando itinerarios, gestión de gastos, información climática y recomendaciones de actividades en un solo lugar. Después de analizar la magnitud del proyecto, se decidió delimitarlo solo a América Latina (LATAM).

Dentro de la elaboración de la página web de WiseTrip, una de las conexiones más importantes son las pasarelas de pago, ya que estas se encargan de procesar las transacciones de las comisiones por reservas de actividades, hospedaje y servicios turísticos ofrecidos dentro de la página, además de las transacciones de los itinerarios personalizados. Por este motivo se nos hace pertinente investigar y comparar opciones de APIs de pasarelas de pago, con el fin de seleccionar la que mejor se ajuste a los requerimientos del negocio de WiseTrip.

Objetivo de la investigación

Identificar y comparar las diferentes pasarelas de pago que tengan dominio sobre América Latina que ofrezcan una API de integración, se evaluará la cobertura geográfica, la estructura de comisiones, los métodos de pago soportados y la facilidad de integración en una página web, para poder escoger la más adecuada para las funcionalidades de WiseTrip.

Criterios de evaluación

Para comparar las opciones de pasarelas de pago se definieron los siguientes criterios:

- Cobertura de países en América Latina: los países donde opera de manera directa.
- Comisiones: porcentaje cobrado por transacción y costos fijos asociados.
- Facilidad de integración: existencia en entorno sandbox y SDKs
- Métodos de pago locales soportados: tarjetas, PSE, transferencias.
- Seguridad: cumplimiento de estándares como PSI, DSS y manejo de webhooks.

Análisis comparativo de pasarelas de pago en LATAM

Mercado Pago

Mercado Pago es la plataforma de pagos digitales más grande de América Latina, parte del ecosistema de Mercado Libre, con presencia en Argentina, Brasil, México, Chile, Colombia, Perú y Uruguay (GuiaDeBancos, 2026a). Ofrece una API completa ya que permite cobros con tarjeta de crédito, débito, código QR y transferencias. Mercado Pago cuenta con un entorno de pruebas (sandbox) al alcance de los desarrolladores. Por otro lado, las comisiones varían según el país y el método de pago; en 2026 la acreditación al día siguiente ronda entre el 2,99 % y el 3,99 % más impuestos según el país, mientras que el cobro por código QR resulta considerablemente más económico (GuiaDeBancos, 2026b). Sus ventajas se basan en el soporte y reconocimiento que tiene gracias al respaldo que tiene de Mercado Libre, su reconocimiento en la región, la alta aprobación de transacciones y el monitoreo antifraude. Su única desventaja es que requiere la creación de una cuenta por cada país en el que se desee operar.

Wompi

Wompi es la pasarela de pagos del grupo Bancolombia, lanzada en 2020 con enfoque en el mercado colombiano. La API que maneja Wompi es API REST, esta permite tres modalidades de integración: widget integrado en el sitio web, redirección del checkout de Wompi, o integración directa vía API para construir un checkout propio. Soporta métodos de pago usados en Colombia, como tarjetas, PSE, Nequi, Botón Bancolombia y correspondientes bancarios. Su documentación técnica incluye entorno sandbox, manejo de webhooks para eventos de transacción y soporte de idempotencia para evitar pagos duplicados (Wompi, s.f.). Las comisiones rondan el 2.99% más IVA para tarjetas, sin cuota mensual fija. Su limitación es que opera únicamente en pesos colombianos, por lo que no tiene cobertura completa en los países de LATAM.

PayU (Rapyd)

PayU es una pasarela de pagos con presencia en siete países de la región: Brasil, Argentina, México, Chile, Colombia, Perú y Uruguay. . En marzo de 2025 su negocio de pagos en Latinoamérica y África fue adquirido por Rapyd, aunque la plataforma y la API continúan operando con normalidad mientras se hace en cambio de marca. La API que utiliza maneja prácticas de seguridad del estandar PCIDSS (Payment Card Industry Data Security Standard), el cual define cómo debe manejarse, almacenarse y tramitar la información de las tarjetas crédito/débito para evitar robos o pérdida de datos. Otras de sus ventajas es que soporta múltiples métodos de pago según el país (tarjetas, transferencias bancarias) y permite personalizar las comisiones según el volumen de transacciones. En Colombia, por ejemplo, la tarifa estándar es de 3,29% más un cargo fijo por transacción.

Tabla 1 Comparación entre tres APIs

Criterio	Mercado Pago	Wompi	PayU
Cobertura LATAM	Argentina, Brasil, México, Chile, Colombia, Perú, Uruguay	Colombia (moneda COP únicamente)	Brasil, Argentina, México, Chile, Colombia, Perú, Uruguay
Comisión aproximada	~2,99 % a 3,99 % + IVA según país y método	~2,99 % + IVA (tarjetas), sin cuota mensual	~3,29 % + cargo fijo (varía por país)
Métodos de pago	Tarjetas, QR, transferencias	Tarjetas, PSE, Nequi, Botón Bancolombia, efectivo	Tarjetas, transferencias, efectivo (según país)
Documentación / sandbox	Amplia, con sandbox oficial	Clara y moderna, con sandbox y webhooks	Disponible, orientada a integración HTTPS/SSL

Limitación principal	Requiere cuenta por país	Solo opera en Colombia (COP)	Transición de marca a Rapyd en curso
----------------------	--------------------------	------------------------------	--------------------------------------

Recomendación para WiseTrip

Trayendo a consideración que WiseTrip esta pensado en usuarios de América Latina y que como equipo se trabaja desde Colombia, se recomienda iniciar las pruebas con Mercado Pago, ya que es el que tiene mayor cobertura, disponibilidad de entorno sandbox y es conocido entre los usuarios que se espera usen la página web. Al utilizar Mercado Pago, se ahorra el proceso de contar con una entidad legal en cada país, algo que si exige Wompi o Stripe. Cuando el proyecto llegue a mayores alcances, podría evaluarse la integración de una API adicional como PayU para optimizar comisiones en mercados específicos.

Integración de Mercado Pago (Checkout Pro) en WiseTrip

Guía base para conectar la pasarela de pagos al backend de WiseTrip y mostrar el checkout desde el frontend.

1. Prerrequisitos

1. Crear cuenta en <https://www.mercadopago.com.co/developers> (o el dominio del país que usen para pruebas).
2. En Tus integraciones → Crear aplicación, obtener las credenciales de prueba:
 - ACCESS_TOKEN (privado, va en el backend)
 - PUBLIC_KEY (público, solo si usan bricks/checkout embebido — para JavaFX no la necesitan)
3. Crear usuarios de prueba (comprador y vendedor) desde el panel de developers para simular pagos sin plata real.

2. Backend con Spring Boot

2.1. Dependencia Maven

```
<dependency>
    <groupId>com.mercadopago</groupId>
    <artifactId>sdk-java</artifactId>
    <version>2.8.0</version>
</dependency>
```

2.2. Configuración — application.properties

```
server.port=8080
mercadopago.access-token=TEST-

mercadopago.notification-
url=https://TU_NGROK_O_DOMINIO/api/pagos/webhook
```

2.3. Configurar el cliente — MercadoPagoConfig.java

```
package com.wisetrip.pagos.config;

import jakarta.annotation.PostConstruct;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Component;

@Component
public class MercadoPagoConfig {

    @Value("${mercadopago.access-token}")
    private String accessToken;

    @PostConstruct
```

```

        public void init() {
            // Usa el nombre completo (fully qualified) para no
chocar con el
            // nombre de esta misma clase

com.mercadopago.MercadoPagoConfig.setAccessToken(accessToken);
        }
    }

```

2.4. Endpoint para crear la preferencia de pago — PagoController.java

```

package com.wisetrip.pagos.controller;

import com.mercadopago.client.preference.*;
import com.mercadopago.resources.preference.Preference;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.web.bind.annotation.*;

import java.util.Collections;
import java.util.List;

@RestController
@RequestMapping("/api/pagos")
public class PagoController {

    @Value("${mercadopago.notification-url}")
    private String notificationUrl;

    public record SolicitudPago(String itinerarioId, String
descripcion, Double monto, String usuarioEmail) {}

    @PostMapping("/crear-preferencia")

```

```

        public Object crearPreferencia(@RequestBody SolicitudPago
solicitud) {
            try {
                PreferenceItemRequest item =
PreferenceItemRequest.builder()
                    .id(solicitud.itinerarioId())
                    .title(solicitud.descripcion())
                    .quantity(1)

.unitPrice(java.math.BigDecimal.valueOf(solicitud.monto()))
                    .currencyId("COP") // ajustar según país
                    .build();

                PreferencePayerRequest payer =
PreferencePayerRequest.builder()
                    .email(solicitud.usuarioEmail())
                    .build();

                PreferenceRequest request =
PreferenceRequest.builder()
                    .items(List.of(item))
                    .payer(payer)
                    .externalReference(solicitud.itinerarioId())
                    .notificationUrl(notificationUrl)
                    .build();

                PreferenceClient client = new PreferenceClient();
                Preference preference = client.create(request);

                return Collections.singletonMap("init_point",
preference.getInitPoint());
            } catch (Exception e) {

```



```

        e.printStackTrace();
        throw new RuntimeException("No se pudo crear la
preferencia de pago");
    }
}
}

```

2.5. Webhook — recibir la confirmación real del pago

```

import com.mercadopago.client.payment.PaymentClient;
import com.mercadopago.resources.payment.Payment;
import java.util.Map;

@PostMapping("/webhook")
public void webhook(@RequestBody Map<String, Object> body) {
    try {
        String type = (String) body.get("type");

        if ("payment".equals(type)) {
            Map<String, Object> data = (Map<String, Object>)
body.get("data");
            Long paymentId = Long.valueOf((String)
data.get("id"));

            PaymentClient paymentClient = new PaymentClient();
            Payment pago = paymentClient.get(paymentId);

            // pago.getStatus() puede ser: approved, pending,
rejected, in_process...
            que enviaste al crear la preferencia

```

```

        System.out.println("Pago " + paymentId + " ->
estado: " + pago.getStatus());

        // TODO: actualizar el estado de la transacción en
la base de datos
        //
transaccionRepository.actualizarEstado(pago.getExternalReference
(), pago.getStatus());
    }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

2.6. Endpoint de estado — para que la app JavaFX consulte por polling

```

@GetMapping("/estado/{itinerarioId}")
public Map<String, String> consultarEstado(@PathVariable String
itinerarioId) {
    String estado = "pending"; // reemplazar por la consulta
real a la BD
    return Collections.singletonMap("estado", estado);
}

```

3. Frontend en JavaFX

3.1. Llamar al backend para crear la preferencia

```

import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

```

```

public class PagoService {

    private final HttpClient client =
HttpClient.newHttpClient();

    private static final String BACKEND_URL =
"http://localhost:8080/api/pagos";

    public String crearPreferencia(String itinerarioId, String
descripcion, double monto, String email) throws Exception {
        String jsonBody = ""
        {
            "itinerarioId": "%s",
            "descripcion": "%s",
            "monto": %.2f,
            "usuarioEmail": "%s"
        }
        """".formatted(itinerarioId, descripcion, monto,
email);

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BACKEND_URL + "/crear-preferencia"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(jsonBody))
            .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());

        // Ejemplo simple sin librería, asumiendo alguna
respuesta {"init_point":"https://..."}
        String body = response.body();

```

```

        return
body.split("\"init_point\":\") [1].split("\") [0];
    }

    public String consultarEstado(String itinerarioId) throws
Exception {
        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create(BACKEND_URL + "/estado/" +
itinerarioId))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
HttpResponse.BodyHandlers.ofString());
        return
response.body().split("\"estado\":\") [1].split("\") [0];
    }
}

```

3.2. Abrir el checkout en el navegador del sistema

```

import java.awt.Desktop;
import java.net.URI;

public void abrirCheckout(String initPoint) {
    try {
        if (Desktop.isDesktopSupported() &&
Desktop.getDesktop().isSupported(Desktop.Action.BROWSE)) {
            Desktop.getDesktop().browse(new URI(initPoint));
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}

```

```
    }  
}
```

3.3. Controlador de la vista — botón "Pagar" + polling del estado

```
import javafx.application.Platform;  
import javafx.fxml.FXML;  
import javafx.scene.control.Button;  
import javafx.scene.control.Label;  
  
import java.util.concurrent.Executors;  
import java.util.concurrent.ScheduledExecutorService;  
import java.util.concurrent.TimeUnit;  
  
public class CheckoutController {  
  
    @FXML private Button botonPagar;  
    @FXML private Label estadoLabel;  
  
    private final PagoService pagoService = new PagoService();  
    private ScheduledExecutorService scheduler;  
  
    @FXML  
    private void onPagarClick() {  
        String itinerarioId = "itin-001";  
  
        try {  
            String initPoint = pagoService.crearPreferencia(  
                itinerarioId, "Tour Ciudad Perdida", 250000.0,  
                "usuario@correo.com"  
            );  
  
            abrirCheckout(initPoint);  
        }  
    }  
}
```

```

        estadoLabel.setText("Esperando confirmación del
        pago...");

        iniciarPolling(itinerarioId);

    } catch (Exception e) {
        estadoLabel.setText("Error al iniciar el pago");
        e.printStackTrace();
    }
}

private void iniciarPolling(String itinerarioId) {
    scheduler =
Executors.newSingleThreadScheduledExecutor();
    scheduler.scheduleAtFixedRate(() -> {
        try {
            String estado =
        pagoService.consultarEstado(itinerarioId);

            if (!estado.equals("pending")) {
                Platform.runLater(() -> {
                    estadoLabel.setText("Estado del pago: "
+ estado);

                    if (estado.equals("approved")) {
                        // mostrarPantallaExito();
                    }
                });
            }
            scheduler.shutdown();
        }
    } catch (Exception e) {
        e.printStackTrace();
    }
}, 3, 3, TimeUnit.SECONDS);

```

```

    }

    private void abrirCheckout(String initPoint) {
        try {
            java.awt.Desktop.getDesktop().browse(new
java.net.URI(initPoint));
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

```

4. Probar en sandbox

1. Levantar el backend (./mvnw spring-boot:run) y exponerlo con ngrok si vamos a probar el webhook.
2. Correr la app JavaFX y hacer clic en "Pagar", debería abrirse el navegador con el checkout de Mercado Pago.
3. Iniciar sesión con el usuario de prueba comprador que creamos en el paso de prerequisites.
4. Usar una tarjeta de prueba
5. Verificar en la consola del backend que el webhook llegó, y que la app JavaFX detecta el cambio de estado.

Conclusión

La elección para el proyecto de WiseTrip es Mercado Pago por su cobertura, la facilidad de integración y sus costos de comisión, por ser un proyecto limitado a solo LATAM. De las tres opciones que se analizaron en este documento, Mercado Pago cumple con los criterios que se estaban buscando, Wompi y PayU por el otro lado, serán consideradas viables dependiendo el crecimiento del proyecto y si se considera pertinente la implementación de una API adicional.

Referencias

GuiaDeBancos. (2026a, 2 de abril). *Mercado Pago: análisis completo 2026 — ¿La mejor fintech en LATAM?* <https://www.guiadebancos.com/banco/mercadopago>

GuiaDeBancos. (2026b, 26 de mayo). *Comisiones Mercado Pago 2026 por país: Chile, Argentina, México, Colombia, Uruguay y Perú.* <https://www.guiadebancos.com/ar/blog/comisiones-mercado-pago-latam-2026>

Mercado Pago. (s. f.-a). *Configurar notificaciones de pago.* Mercado Pago Developers. <https://www.mercadopago.com.co/developers/es/docs/checkout-pro/payment-notifications>

Mercado Pago. (s. f.-b). *Crear y configurar una preferencia de pago.* Mercado Pago Developers. <https://www.mercadopago.com.co/developers/es/docs/checkout-pro/create-payment-preference>

Mercado Pago. (s. f.-c). *Developers.* <https://www.mercadopago.com/developers>

Mercado Pago. (s. f.-d). *Información adicional sobre notificaciones.* Mercado Pago Developers. <https://www.mercadopago.com.co/developers/es/docs/checkout-pro/additional-content/notifications/additional-info>

Mercado Pago. (s. f.-e). *Webhooks.* Mercado Pago Developers. <https://www.mercadopago.com.co/developers/es/docs/checkout-pro/additional-content/notifications/webhooks>

PCI Security Standards Council. (s. f.). *PCI DSS.* <https://www.pcisecuritystandards.org/standards/pci-dss/>

Rapyd. (2025, 14 de marzo). *Rapyd completes acquisition of PayU Global Payment Organisation in Latin America and Africa.* <https://www.rapyd.net/company/news/press-releases/rapyd-completes-acquisition-of-payu-latin-america-and-africa/>

Rapyd. (s. f.). *Rapyd API, integration and developer resources*.
<https://www.rapyd.net/developers/>

Wompi. (s. f.). *Referencia del API*. Recuperado el 21 de agosto de 2026, de
<https://docs.wompi.co/docs/colombia/referencia-pagos-a-terceros/>